VULNERABILITY TESTING / LAB REPORT

Authentication Vulnerabilities

Web Application Penetration Testing (WAPT) Internship, Phase 1

IqraSec Academy

Field	Details
Intern Name	Rohit Kushwah
Organization	IqraSec Academy
Internship	Web Application Penetration Testing (WAPT), Phase 1
Topic	Authentication Vulnerabilities
Platform	TryHackMe
Lab Name	Authentication Bypass
Number of Labs	1
Completion Date	25-10-2026

Template note: Delete all italic instructions and replace placeholders before submission. Include actual lab results and genuine screenshots. Write in your own words and use first person.

Lab 1: Authentication Bypass

1. Lab Name

Authentication Bypass — TryHackMe

Room URL: https://tryhackme.com/room/authenticationbypass

2. Objective

The objective of this lab is to understand authentication vulnerabilities and learn how weaknesses in login mechanisms can allow unauthorized users to access restricted functionality.

I studied common authentication weaknesses, including username enumeration, weak authentication logic, insecure password reset processes, and cookie tampering. I also reviewed ways to identify authentication weaknesses and how developers can prevent them.

3. Vulnerability / Concept

Vulnerability: Authentication Vulnerabilities

Authentication is the process by which an application verifies a user's identity. Authentication vulnerabilities occur when this process can be bypassed, manipulated, or abused because of insecure implementation.

Common types of authentication vulnerabilities:

- **Authentication Bypass:** Accessing restricted functionality without correctly completing the required authentication process.
- **Username Enumeration:** Identifying valid usernames through differences in application responses.
- **Weak Password Protection:** Allowing weak passwords or insufficient protection against repeated login attempts.
- **Logic Flaws:** Exploiting errors in login, registration, or password reset workflows.
- **Cookie Tampering:** Modifying authentication-related cookies when the application fails to validate them securely.
- **Insecure Session Management:** Mishandling session identifiers, expiration, or session invalidation.

Why it happens: Authentication vulnerabilities may result from insecure application logic, inadequate server-side validation, weak password policies, or incorrect handling of session data.

Where it appears: Login pages, registration forms, password reset workflows, multi-factor authentication processes, and session management mechanisms.

Security impact: Depending on the weakness, an attacker may gain unauthorized account access, impersonate another user, or access protected application functionality.

4. Testing Approach

The following steps describe the planned testing workflow for the Authentication Bypass room. Update these steps to match the tests actually performed in the lab.

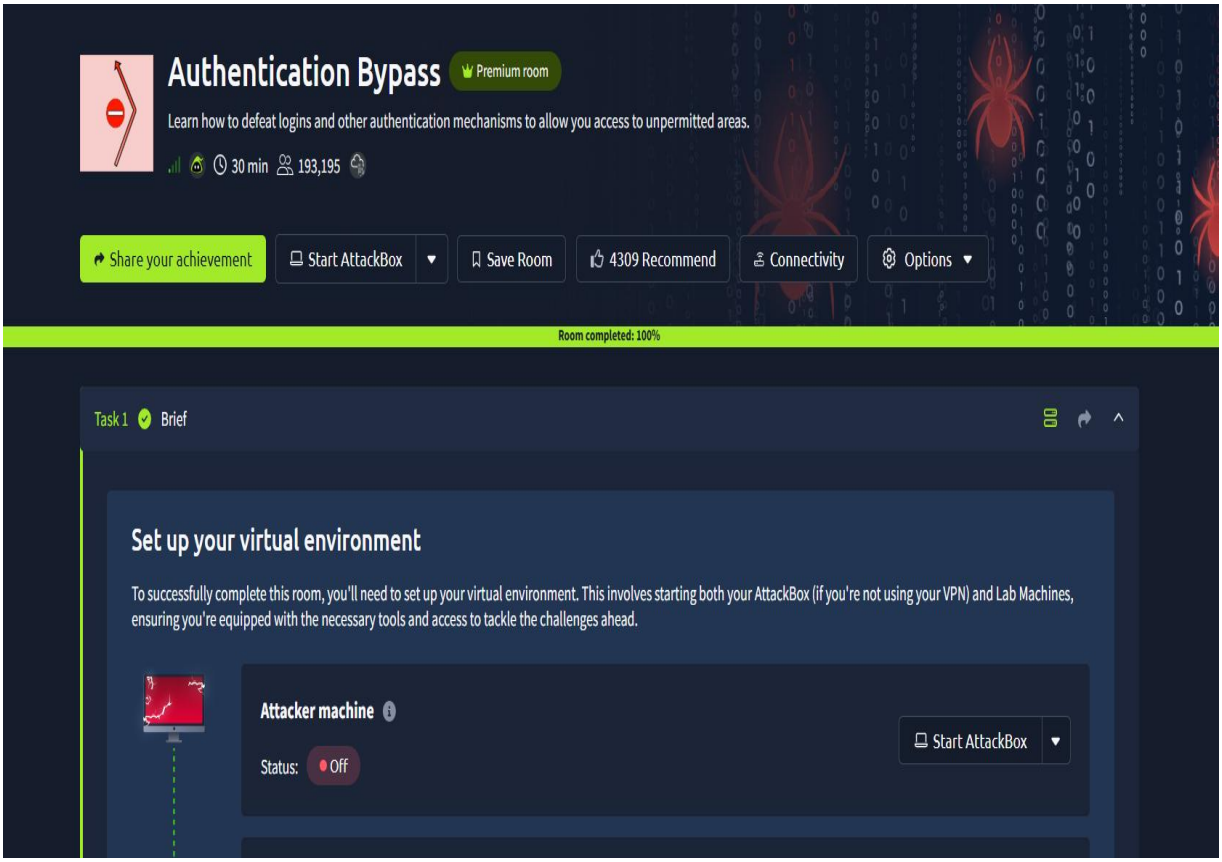
1. Opened the TryHackMe Authentication Bypass room.
2. Reviewed the room introduction and learning objectives.
3. Started the authorized lab environment, if required.
4. Identified the login page and available authentication mechanisms.
5. Observed the normal login behavior using valid or invalid test credentials provided by the lab.
6. Compared application responses for different authentication outcomes.
7. Reviewed username enumeration as a possible authentication weakness.
8. Examined how an application might respond to unknown usernames.
9. Studied the risks associated with weak login protection.

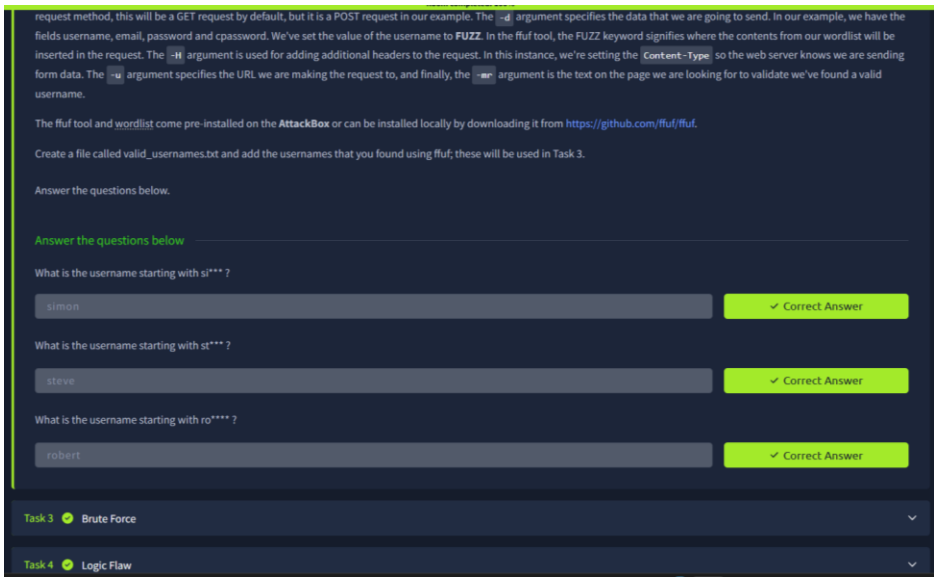
10. Reviewed authentication logic and possible validation errors.
11. Examined the security of password reset workflows.
12. Studied how inconsistent parameter handling could affect authentication.
13. Reviewed authentication-related cookies used by the application.
14. Studied the risks of trusting client-controlled authentication data.
15. Reviewed server-side verification of authentication state.
16. Examined session handling and the importance of secure session validation.
17. Reviewed the potential impact of successful authentication bypass.
18. Studied recommended defenses for authentication mechanisms.
19. Recorded relevant observations and captured genuine screenshots.
20. Verified the lab result and completion status, if the practical exercise was completed.

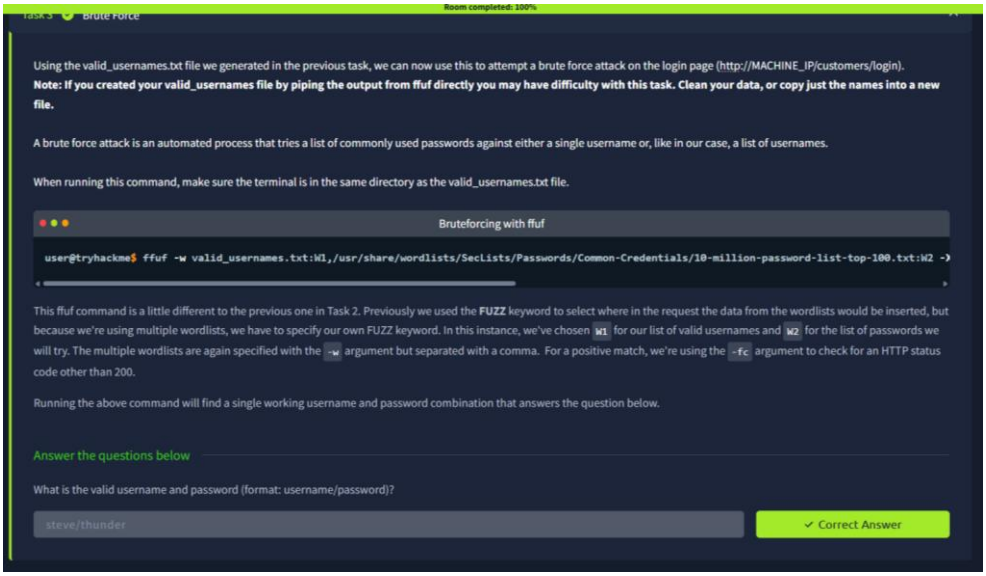
Payload / HTTP Request Used, if any:

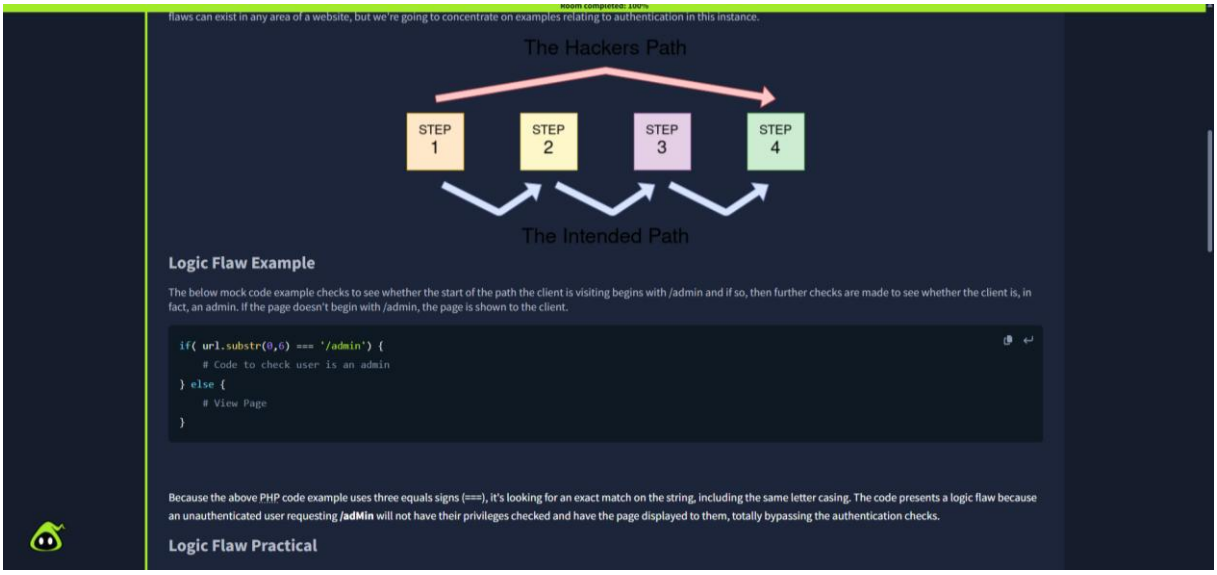
[Insert the actual request, test input, or observation from the authorized lab. Do not add untested payloads or invented results.]

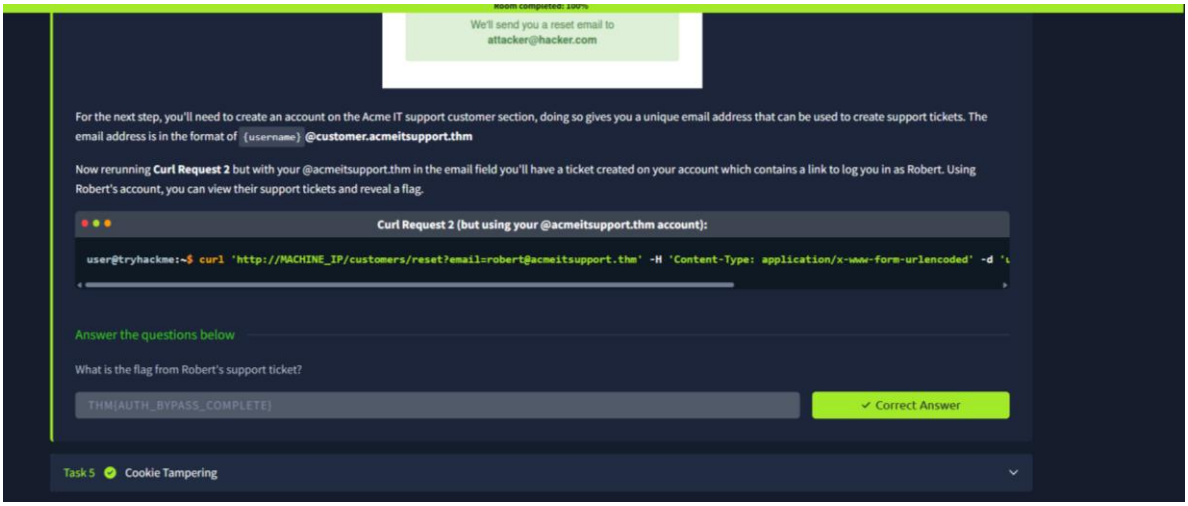
5. Evidence / Screenshots

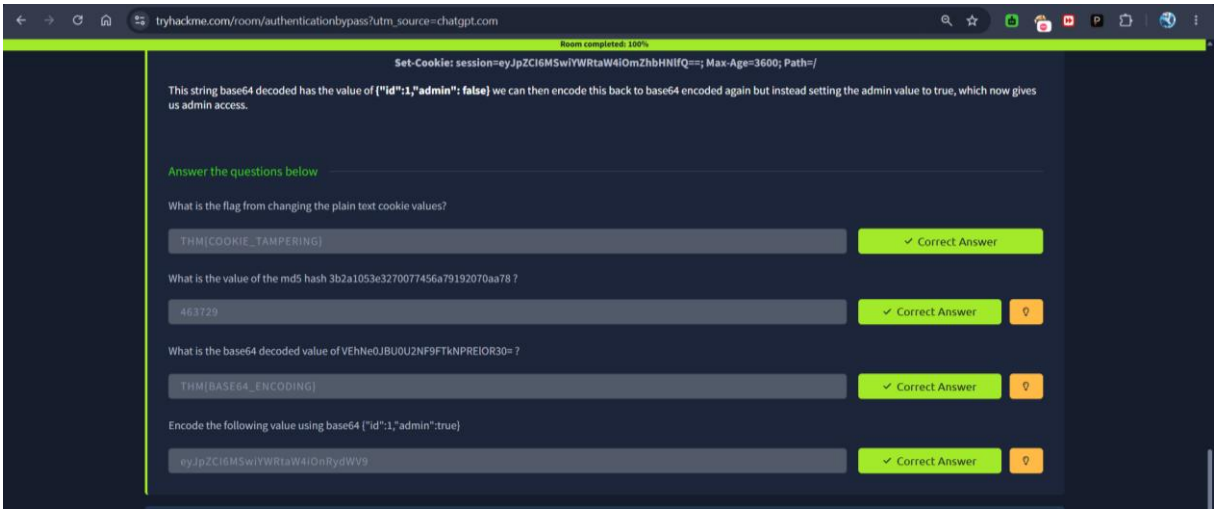



request method, this will be a GET request by default, but it is a POST request in our example. The -d argument specifies the data that we are going to send. In our example, we have the fields username, email, password and cpassword. We've set the value of the username to FUZZ. In the ffuf tool, the FUZZ keyword signifies where the contents from our wordlist will be inserted in the request. The -H argument is used for adding additional headers to the request. In this instance, we're setting the Content-Type so the web server knows we are sending form data. The -u argument specifies the URL we are making the request to, and finally, the -mr argument is the text on the page we are looking for to validate we've found a valid username.
The ffuf tool and wordlist come pre-installed on the AttackBox or can be installed locally by downloading it from https://github.com/ffuf/ffuf.
Create a file called valid_usernames.txt and add the usernames that you found using ffuf; these will be used in Task 3.
Answer the questions below.
Answer the questions below
What is the username starting with si*** ?
simon
✓ Correct Answer
What is the username starting with st*** ?
steve
✓ Correct Answer
What is the username starting with ro**** ?
robert
✓ Correct Answer
Task 3 Brute Force
Task 4 Logic Flaw


Room completed: 100%
Using the valid_usernames.txt file we generated in the previous task, we can now use this to attempt a brute force attack on the login page (http://MACHINE_IP/customers/login).
Note: If you created your valid_usernames file by piping the output from ffuf directly you may have difficulty with this task. Clean your data, or copy just the names into a new file.
A brute force attack is an automated process that tries a list of commonly used passwords against either a single username or, like in our case, a list of usernames.
When running this command, make sure the terminal is in the same directory as the valid_usernames.txt file.
Bruteforcing with ffuf
user@tryhackme$ ffuf -w valid_usernames.txt:W1,/usr/share/wordlists/SecLists/Passwords/Common-Credentials/10-million-password-list-top-100.txt:W2 -
This ffuf command is a little different to the previous one in Task 2. Previously we used the FUZZ keyword to select where in the request the data from the wordlists would be inserted, but because we're using multiple wordlists, we have to specify our own FUZZ keyword. In this instance, we've chosen W1 for our list of valid usernames and W2 for the list of passwords we will try. The multiple wordlists are again specified with the -w argument but separated with a comma. For a positive match, we're using the -fc argument to check for an HTTP status code other than 200.
Running the above command will find a single working username and password combination that answers the question below.
Answer the questions below
What is the valid username and password (format: username/password)?
steve/thunder
✓ Correct Answer


Room completed: 100%
flaws can exist in any area of a website, but we're going to concentrate on examples relating to authentication in this instance.
The Hackers Path
STEP 1
STEP 2
STEP 3
STEP 4
The Intended Path
Logic Flaw Example
The below mock code example checks to see whether the start of the path the client is visiting begins with /admin and if so, then further checks are made to see whether the client is, in fact, an admin. If the page doesn't begin with /admin, the page is shown to the client.
if(url.substr(0,6) === '/admin') {
Code to check user is an admin
} else {
View Page
}
Because the above PHP code example uses three equals signs (===), it's looking for an exact match on the string, including the same letter casing. The code presents a logic flaw because an unauthenticated user requesting /adMin will not have their privileges checked and have the page displayed to them, totally bypassing the authentication checks.
Logic Flaw Practical





6. Impact

Authentication vulnerabilities can have serious consequences depending on the weakness and the application's security controls.

Potential impacts include:

- Unauthorized access to user accounts.
- Impersonation of legitimate users.
- Access to restricted application features.
- Exposure of personal or confidential information.
- Abuse of password reset or account recovery functionality.
- Session-related attacks when authentication state is not securely managed.

The actual impact must be based on the results demonstrated in the authorized lab. A suspected weakness should not be reported as a confirmed account compromise without supporting evidence.

7. Remediation

- **Secure authentication logic:** Enforce authentication checks on the server for every protected request.
- **Use strong password storage:** Store passwords using a suitable password-hashing algorithm with appropriate salts and parameters.
- **Prevent username enumeration:** Use consistent responses for invalid usernames and passwords where appropriate.
- **Implement rate limiting:** Limit repeated authentication attempts and monitor suspicious login activity.
- **Secure password reset workflows:** Use cryptographically secure, time-limited, single-use reset tokens.
- **Validate session state:** Do not trust client-controlled values as proof of authentication.
- **Protect cookies:** Use appropriate HttpOnly, Secure, and SameSite attributes for session cookies.
- **Regenerate session identifiers:** Regenerate session IDs after login and other relevant privilege changes.
- **Implement multi-factor authentication:** Add MFA for sensitive accounts and actions where appropriate.
- **Log security events:** Monitor suspicious login attempts and authentication failures without logging passwords or session secrets.

8. What You Learned

- I learned the purpose of authentication in web applications.
- I understood how authentication bypass can occur because of implementation flaws.
- I studied username enumeration and weak authentication logic.
- I learned why password reset workflows require careful validation.
- I understood the risks associated with insecure cookies and session management.
- I reviewed the importance of server-side authentication checks.
- I learned about rate limiting, secure password storage, MFA, and session protection.
- I understood why genuine testing evidence is necessary to support security findings.

Conclusion: This report documents the concepts and testing workflow for authentication vulnerabilities using the TryHackMe Authentication Bypass room. The practical findings and final completion status must be updated using the results and screenshots from the actual lab session.